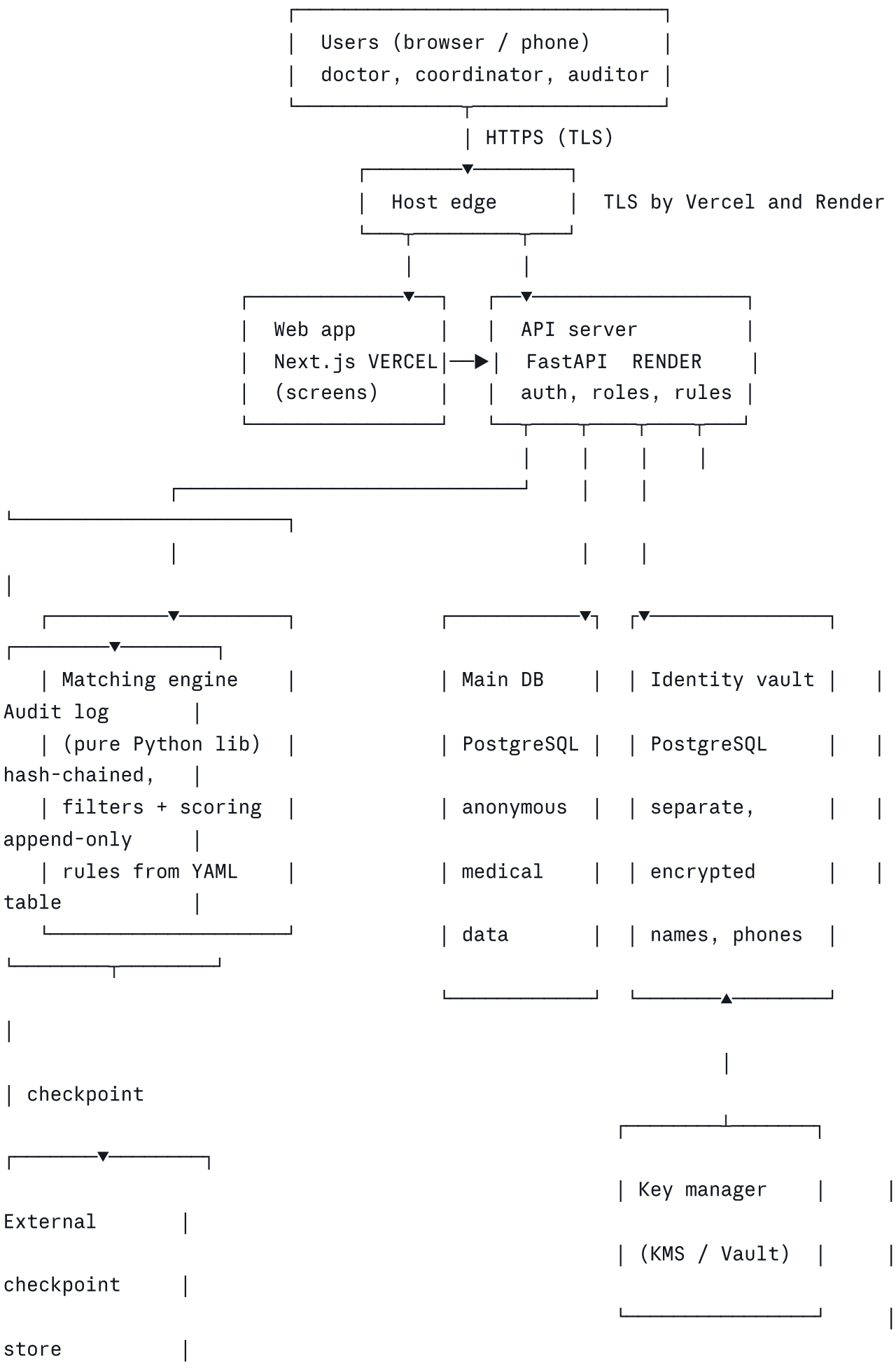
PraanSetu: Architecture

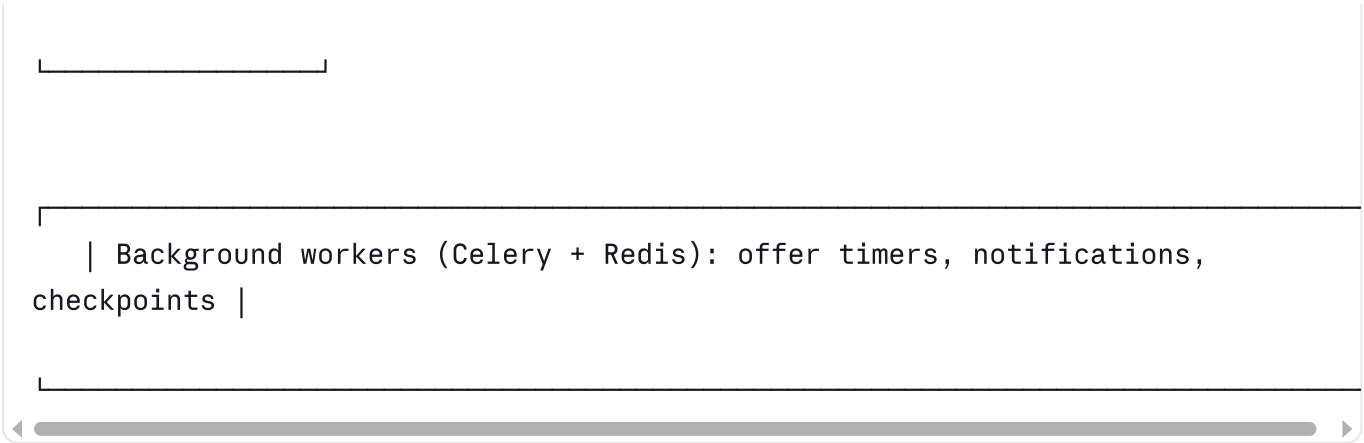
This document explains how PraanSetu is built: the big picture, the technology stack, how each part works, and the full folder structure of the project. It follows the PRD and README.

> Status: planning and prototype stage. Use fake data only until legal and medical review is done.

---

1. Architecture in one picture





How to read it. The web app is hosted on Vercel; the API, workers, databases and Redis are hosted on Render. The browser talks only to the web app and API. The API is the only part that touches the databases. The matching engine is a separate library with no database access, so it is easy to test and audit. The identity vault is a different database with different keys, so one leak does not reveal everything.

---

2. Technology stack

Layer	Choice	Why
Front end	Next.js + TypeScript	Fast pages, strong typing, easy role-based screens
Styling	Tailwind CSS with custom design tokens (no UI kit defaults)	Gives the calm, non-template look in the PRD
Tables and forms	TanStack Table, React Hook Form, Zod	Tables are the core screen; strict form checking
Front end state and data	TanStack Query	Simple server data and caching
Fonts	A serif for headings, a sans for body, a mono for anonymous codes (self-hosted)	Readable, and codes like `R-4F7A91` are easy to compare
API server	Python 3.12 + FastAPI	Clear, fast, automatic API docs
Data checks	Pydantic v2	Strict input and output rules
Database access	SQLAlchemy 2 + Alembic migrations	Safe queries and versioned schema changes
Main database	PostgreSQL 16	Reliable, supports row-level security and triggers
Identity vault	Separate PostgreSQL instance, field-level encryption	Keeps real identity apart from medical data
Background jobs	Celery + Redis	Offer countdowns, notifications, log checkpoints

Layer	Choice	Why
Authentication	JWT sessions + TOTP multi-factor (pyotp), password hashing with Argon2	Strong login without heavy setup
Permissions	Role-based access control (RBAC) in the API, plus Postgres row-level security	Two layers of protection
Encryption	TLS in transit, AES-256 at rest, envelope encryption with an external key manager (AWS KMS, Mumbai region); Render only holds the access credentials as secrets	Standard protection for health data
Audit log	Append-only hash-chained table (SHA-256)	Tamper detection
Notifications	Email (SMTP or SES) and SMS gateway, plus in-app	Offer alerts
Edge and TLS	Vercel (frontend) and Render (backend) give HTTPS automatically; security headers set in `next.config.js` and the API; rate limits in the API (slowapi)	No server of our own to manage
Containers	Docker + Docker Compose for local work; Render runs the API from a Dockerfile	Same setup locally and online
Frontend hosting	**Vercel** (Next.js), server region set to Mumbai (`bom1`)	Built for Next.js, preview link for every change
Backend hosting	**Render**: Web Service (API), Background Worker (Celery), Cron Job (scheduled tasks), two Postgres databases (main and vault), Key Value store (Redis), all described in one `render.yaml`	One place for everything behind the API
CI/CD	GitHub Actions runs tests and security scans; Vercel and Render auto-deploy from GitHub only after the checks pass	Nothing untested goes live
Infrastructure code	`render.yaml` (Render Blueprint) and `vercel.json`	Repeatable setup without Terraform
Monitoring	Render and Vercel built-in logs and metrics, Sentry for errors, an uptime monitor with alerts, structured JSON logs	See problems early (no personal data in logs)
Testing	pytest (back end), Vitest + Testing Library (front end), Playwright (end-to-end), Hypothesis (rule property	Full coverage of the rules

Layer	Choice	Why
	tests)	
Security scans	Bandit, pip-audit, npm audit, Trivy, OWASP ZAP	Find known weaknesses
Docs	MkDocs or plain Markdown	Simple

Later phases: HL7 FHIR adapter for hospital systems, a permissioned blockchain (Hyperledger Fabric) if several organizations need shared trust, maps and routing service for travel time, Hindi and Marathi translations.

---

3. The main parts and what each one does

3.1 Web app (`apps/web`)

- Screens: Sign in, Home, Donors, Recipients, Match results, Offer detail, Committee review, Audit log, Reports, Admin.
- Never stores medical data in the browser beyond the current page.
- Shows "Identity hidden" labels and asks for a reason before any reveal.

3.2 API server (`apps/api`)

- Checks login and role on every request.
- Saves donors, recipients, offers and decisions.
- Calls the matching engine, then stores the result and writes the audit entry in one transaction.
- Is the only code allowed to talk to the identity vault.

3.3 Matching engine (`packages/matching-engine`)

- A plain Python library with **no database and no network**.
- Input: one donor and a list of recipients (anonymous codes only).
- Step 1: hard filters (organ, blood group, size, crossmatch, status).
- Step 2: scoring (urgency, HLA match, waiting time, distance, pediatric priority).
- Output: ranked list, plus a plain-language reason for each rank and the rule version used.
- Rules and weights are in versioned YAML files approved by the medical board. Same input always gives the same output.

3.4 Identity vault (`apps/api/app/vault`)

- Stores names, phone numbers, addresses and government IDs, encrypted per field.
- Linked to the main database only by the anonymous code.
- A reveal needs a reason and a second approval, and writes an audit entry.

3.5 Audit log (`apps/api/app/audit`)

- Each entry holds: id, time, actor code, action, payload fingerprint, previous entry fingerprint, and its own fingerprint.
- Own fingerprint = SHA-256 of (previous fingerprint + canonical payload).
- A database rule blocks UPDATE and DELETE on the table.
- A "Verify log" job walks the whole chain and reports pass or fail.
- A scheduled job sends the latest fingerprint to an outside store, so even an operator cannot quietly rewrite history.

3.6 Workers (`apps/api/app/workers`)

- Offer timers: when an offer expires, move to the next candidate.
- Notifications: send email, SMS and in-app alerts.
- Checkpoints: publish log fingerprints.
- Reports: nightly fairness and waiting-time numbers.

---

4. How one match flows through the system

1. **Coordinator** lists a donor organ in the web app.
2. **API** checks login, role and input, saves the donor under an anonymous code.
3. **API** loads waiting recipients (anonymous data only) and calls the **matching engine**.
4. **Engine** filters, scores and returns a ranked list with reasons.
5. **API** saves the match run and writes an **audit entry** (same transaction).
6. **Worker** sends an offer to the top candidate's doctor and starts a timer.
7. **Doctor** accepts or declines with a reason. The API logs it.
8. If declined or the timer ends, the worker offers the next candidate.
9. **Committee** approves the final match. The API logs it.
10. **Auditor** can click "Verify log" at any time.

---

5. Security design

Area	What we do
Login	Email and password (Argon2) plus mandatory TOTP; short sessions; lock after repeated failures
Access	RBAC in the API, row-level security in PostgreSQL, least privilege for every service account
Identity	Separate vault database, separate keys, reveal needs reason plus approval
Encryption	TLS everywhere; AES-256 at rest; field-level encryption for identity fields; keys in a key manager, rotated on a schedule
Logs	Structured logs with codes only, never names or contact details
Input	Strict validation (Pydantic, Zod); parameterised queries only
Web	Strict Content-Security-Policy, secure cookies, CSRF protection, rate limits; frontend and API on sibling subdomains (`app.` and `api.`) so cookies stay same-site, with a strict CORS allow-list
Dependencies	Automated vulnerability scans in CI; pinned versions
Backups	Encrypted daily backups, tested restore every month
Availability	Health checks, automatic restart, a phone fallback process for urgent offers
Test data	Fake data only; seeds are generated, never copied from real hospitals

6. Data model (simplified)

Table	Database	Key fields
`hospital`	main	id, name, region, verified
`app_user`	main	id, hospital_id, role, mfa_enabled
`donor`	main	donor_code, organ, blood_group, size, test_results, status
`recipient`	main	recipient_code, organ, blood_group, size, hla, urgency, listed_at, status
`match_run`	main	id, donor_code, rule_version_id, created_at, result_json
`offer`	main	id, match_run_id, recipient_code, status, expires_at, responded_at, reason
`decision`	main	id, offer_id, committee_user_id, outcome, reason
`rule_version`	main	id, weights_yaml_hash, approved_by, effective_from
`audit_log`	main (append-only)	id, time, actor_code, action, payload_hash, prev_hash, entry_hash
`identity`	vault	code, name_enc, phone_enc, address_enc, gov_id_enc
`identity_reveal`	vault	id, code, requested_by, approved_by, reason, time

7. API overview (main routes)

Route	Purpose	Allowed roles
`POST /auth/login`, `POST /auth/mfa`	Sign in	everyone
`GET/POST /donors`	List and register donors	coordinator, doctor
`GET/POST /recipients`	List and register recipients	coordinator, doctor
`POST /matches/run`	Run matching for a donor organ	coordinator, doctor
`GET /matches/{id}`	Ranked list with reasons	doctor, committee
`POST /offers/{id}/respond`	Accept or decline with reason	doctor
`POST /decisions`	Committee approval or override	committee
`POST /identity/reveal`	Request identity reveal	doctor (needs approval)
`GET /audit`	Search log entries	auditor, authority
`POST /audit/verify`	Re-check the full chain	auditor, authority
`GET /reports/fairness`	Fairness and waiting-time reports	authority, auditor
`GET/POST /admin/*`	Users, hospitals, rule versions	admin

8. Environments

Environment	Purpose	Data
Local	Developers, Docker Compose	Fake seed data
Staging	Testing and demos (Vercel preview plus a separate Render staging setup)	Fake data only
Pilot	Shadow mode with partner hospitals	Real data only after legal approval
Production	Real use	Real data, full monitoring

8.1 Deployment: Vercel (frontend) and Render (backend)

Part	Where it runs	Notes
Web app (`apps/web`)	Vercel project, root directory `apps/web`	Auto-deploys from `main`; every pull request gets its own preview link; the API address is set as `NEXT_PUBLIC_API_URL`
API (`apps/api`)	Render Web Service (Docker)	Health check at `/health`; deploys after CI passes
Celery worker	Render Background Worker	Same image as the API, different start command
Scheduled jobs (nightly reports, log checkpoints)	Render Cron Job	Runs on a fixed schedule
Main database	Render Postgres	Anonymous medical data and audit log
Identity vault database	A second Render Postgres	Separate login, separate keys, never shared with the main one
Redis	Render Key Value	Job queue for Celery
Secrets	Vercel environment variables and Render environment groups	Never in git; the frontend holds no secrets

Things to get right:

- **Same site for cookies.** Use one domain with two subdomains, for example `app.praansetu.in` (Vercel) and `api.praansetu.in` (Render). Login cookies then stay secure and same-site, and the API accepts requests only from the app domain.
- **No free plans for real use.** Render free services go to sleep when idle and free databases expire. Offers with timers need always-on services, so use paid plans and a paid database with backups.
- **Data location.** When I last checked, Render had no India region (Singapore was closest), while Vercel can run in Mumbai. Confirm Render's current regions, then check with a lawyer whether this is acceptable under the DPDP Act and hospital agreements before any real data. If not, move at least the identity vault and main database to an India-region host later. The code does not change for this.
- **Rollback.** Both platforms keep earlier deployments, so a bad release can be rolled back in one click.

9. Full folder structure

```
praansetu/
├── README.md
├── ARCHITECTURE.md
├── LICENSE
├── .gitignore
├── .env.example                    # example settings, no real secrets
├── docker-compose.yml              # runs everything locally
├── Makefile                        # shortcuts: make dev, make test
│
├── docs/
│   ├── prd.md                      # product requirements
│   ├── matching-rules.md           # rules explained in plain words
│   ├── privacy-and-security.md
│   ├── audit-log-design.md
│   ├── legal-checklist.md          # THOTA, DPDP Act, ABDM, NOTTO notes
│   ├── api/                        # generated API reference
│   ├── design/
│   │   ├── design-principles.md    # calm, clean, not template-looking
│   │   ├── design-tokens.md        # colors, fonts, spacing
│   │   ├── screen-list.md
│   │   └── wireframes/
│   └── runbooks/
│       ├── backup-and-restore.md
│       ├── incident-response.md
│       └── phone-fallback.md
│
├── apps/
│   ├── web/                        # front end (Next.js + TypeScript)
│   │   ├── package.json
│   │   ├── tsconfig.json
│   │   ├── next.config.js
│   │   ├── vercel.json                 # Vercel settings (region, headers)
│   │   ├── tailwind.config.ts
│   │   ├── public/
│   │   │   └── fonts/
│   │   └── src/
│   │       ├── app/                # pages (routes)
│   │       │   ├── (auth)/sign-in/
│   │       │   ├── (app)/home/
│   │       │   ├── (app)/donors/
│   │       │   ├── (app)/recipients/
│   │       │   ├── (app)/matches/
│   │       │   ├── (app)/offers/
```

```
|   |       |   ├── (app)/committee/
|   |       |   ├── (app)/audit/
|   |       |   ├── (app)/reports/
|   |       |   └── (app)/admin/
|   |       ├── components/
|   |       |   ├── ui/              # buttons, inputs, status chips, tables
|   |       |   ├── match/           # ranked table, "why this rank" panel
|   |       |   ├── offer/           # countdown, accept/decline form
|   |       |   └── audit/           # log list, verify button
|   |       ├── lib/                 # api client, auth helpers, formatters
|   |       ├── hooks/
|   |       ├── styles/              # global styles and design tokens
|   |       ├── i18n/                # English now; Hindi and Marathi later
|   |       └── tests/
|   |
|   └── api/                         # back end (FastAPI)
|       ├── pyproject.toml
|       ├── alembic.ini
|       ├── Dockerfile
|       ├── migrations/              # database changes, versioned
|       ├── app/
|       |   ├── main.py              # app start
|       |   ├── config.py            # settings from environment
|       |   ├── deps.py              # shared dependencies
|       |   ├── api/
|       |   |   ├── routes_auth.py
|       |   |   ├── routes_donors.py
|       |   |   ├── routes_recipients.py
|       |   |   ├── routes_matches.py
|       |   |   ├── routes_offers.py
|       |   |   ├── routes_decisions.py
|       |   |   ├── routes_identity.py
|       |   |   ├── routes_audit.py
|       |   |   ├── routes_reports.py
|       |   |   └── routes_admin.py
|       |   ├── core/
|       |   |   ├── security.py      # password hashing, tokens, MFA
|       |   |   ├── permissions.py   # role rules
|       |   |   ├── codes.py         # anonymous code generator
|       |   |   └── errors.py
|       |   ├── models/              # database tables (SQLAlchemy)
|       |   ├── schemas/             # request and response shapes (Pydantic)
|       |   ├── services/
|       |   |   ├── donor_service.py
|       |   |   ├── recipient_service.py
```

```
|       |   |   ├── match_service.py     # calls the matching engine
|       |   |   ├── offer_service.py
|       |   |   ├── decision_service.py
|       |   |   └── report_service.py
|       |   ├── vault/               # identity vault access (only entry
point)
|       |   |   ├── client.py
|       |   |   ├── encryption.py
|       |   |   └── reveal.py
|       |   ├── audit/
|       |   |   ├── chain.py         # build and verify the hash chain
|       |   |   ├── writer.py
|       |   |   └── checkpoint.py
|       |   ├── workers/
|       |   |   ├── celery_app.py
|       |   |   ├── offer_timers.py
|       |   |   ├── notifications.py
|       |   |   ├── checkpoints.py
|       |   |   └── reports.py
|       |   └── db/
|       |       ├── session.py
|       |       └── rls.sql          # row-level security policies
|       ├── scripts/
|       |   ├── seed_fake_data.py    # makes fake donors and recipients
|       |   └── verify_audit_chain.py
|       └── tests/
|           ├── unit/
|           ├── integration/
|           └── security/
|
├── packages/
|   ├── matching-engine/             # pure Python library, no database
|   |   ├── pyproject.toml
|   |   ├── src/praansetu_matching/
|   |   |   ├── __init__.py
|   |   |   ├── models.py            # Donor, Recipient, Result types
|   |   |   ├── filters/
|   |   |   |   ├── blood_group.py
|   |   |   |   ├── size.py
|   |   |   |   ├── crossmatch.py
|   |   |   |   └── status.py
|   |   |   ├── scoring/
|   |   |   |   ├── urgency.py
|   |   |   |   ├── hla.py
|   |   |   |   ├── waiting_time.py
```

```
|   |   |   |   ├── distance.py
|   |   |   |   └── pediatric.py
|   |   |   ├── explain.py           # builds "why this rank" text
|   |   |   ├── engine.py            # runs filters, then scores
|   |   |   └── rules_loader.py      # reads and checks YAML rules
|   |   ├── rules/
|   |   |   ├── kidney/v1.yaml       # first rule set (needs doctor review)
|   |   |   ├── liver/               # later
|   |   |   └── heart/               # later
|   |   └── tests/
|   |       ├── test_filters.py
|   |       ├── test_scoring.py
|   |       ├── test_determinism.py  # same input, same output
|   |       └── cases/               # doctor-checked test cases
|   |
|   └── shared-types/                # shared names and enums (organ, status)
|
├── render.yaml                      # Render Blueprint: API, worker, cron
job, 2 databases, Redis
├── infra/
|   ├── docker/
|   |   └── api.Dockerfile           # builds the backend for Render (also
used locally)
|   └── monitoring/
|       └── uptime-checks.md         # what we watch and who gets alerts
|
├── tools/
|   ├── fake-data-generator/         # realistic fake patients for testing
|   └── audit-verifier/              # standalone checker for auditors
|
└── .github/
    ├── workflows/
    |   ├── ci.yml                   # tests, lint, type checks
    |   ├── security.yml             # dependency and container scans
    |   └── post-deploy-check.yml    # quick health check after Vercel and
Render deploy
    ├── ISSUE_TEMPLATE/
    └── pull_request_template.md
```

Why this layout.

- `apps/` holds things that run (web, API).
- `packages/` holds reusable code. The matching engine lives here so it stays free of databases and is easy for doctors and auditors to review.

- `docs/` keeps the PRD, rules, legal checklist and design notes together.
- `render.yaml` (root) and `apps/web/vercel.json` describe how the backend and frontend are deployed. `infra/` keeps the backend Dockerfile and monitoring notes apart from product code.
- `tools/` holds helper programs, such as the fake data generator and a standalone log checker.

---

10. Build order (what to code first)

1. **Foundation:** repo, Docker Compose, Postgres, FastAPI skeleton, Next.js skeleton, CI. Deploy both skeletons early (frontend on Vercel, backend on Render) so deployment problems show up on day one.
2. **Login and roles:** users, MFA, RBAC, first screens (Sign in, Home).
3. **Registry:** donor and recipient forms, anonymous codes, fake data seeder.
4. **Matching engine:** filters, scoring, kidney `v1.yaml`, tests with doctor-checked cases.
5. **Match results screen:** ranked table with "why this rank" reasons.
6. **Audit log:** hash chain, append-only rule, "Verify log" button.
7. **Offers:** accept or decline, timers, notifications.
8. **Identity vault:** separate database, encryption, reveal with approval.
9. **Committee, reports, admin.**
10. **Security hardening and testing:** scans, penetration test, backup restore test.
11. **Shadow pilot** with partner hospitals after legal approval.

---

11. Key decisions and trade-offs

Decision	Chosen	Alternative	Reason
Back end language	Python (FastAPI)	Node.js	Easy to write and read matching rules; strong data tools
Audit trail	Hash-chained table	Blockchain now	Simple and cheap; add a permissioned chain later
Matching engine	Separate pure library	Inside API	Easy to test, audit and show to doctors
Repo style	Monorepo	Separate repos	One place to change front end, API and rules together
Rules storage	Versioned YAML	Hard-coded	Medical board can review and approve changes
Identity storage	Separate vault database	Same database	Limits damage if one store leaks
First organ	Kidney	All organs	Most patients, clearest rules
Frontend hosting	Vercel	Self-hosted	Made for Next.js; previews and rollbacks included
Backend hosting	Render	Own servers (AWS, Kubernetes)	Simple to run alone; workers, cron, Postgres and Redis in one place

---

12. Open items

- Confirm the final country, state and partner hospitals (this sets the legal rules and rule sets).
- Get a transplant doctor to review `rules/kidney/v1.yaml` before any real use.
- Confirm that Render's regions are acceptable for data-residency rules (see section 8.1), and choose the key manager.
- Buy the domain and plan the `app.` and `api.` subdomains.
- Decide who runs the platform and who owns the audit checkpoints.
- Check the PraanSetu name, domains and trademark.